Acceso a base de datos desde el backend en .NET

Fundamentos de ADO.NET y flujo de conexión

Flujo de conexión

Cuando el backend necesita datos de la base de datos, sigue estos pasos:

1. **Abrir la conexión** con la base de datos (Connection).
2. **Crear el comando** con la instrucción SQL (Command).
3. **Agregar parámetros** si la consulta recibe datos del usuario.
4. **Ejecutar el comando.**
5. **Leer los resultados** con DataReader, o llenar un DataSet con DataAdapter.
6. **Cerrar la conexión.**
7. Si algo falla, **atrapar la excepción** y manejar el error.

App → Connection → Command (+ Parámetros) → Base de datos → DataReader / DataSet → App

1. ADO.NET

Es el conjunto de clases que trae .NET para conectarse a una base de datos, mandarle instrucciones SQL y recibir la información. Es la herramienta base; otras herramientas, como Entity Framework, la usan por debajo.

Trabaja de dos formas:

- **Conectada:** la conexión se mantiene abierta mientras se leen los datos (DataReader).
- **Desconectada:** los datos se copian a la memoria y la conexión se cierra (DataAdapter y DataSet).

2. Proveedor de datos

Es la librería que sabe "hablar" con una base de datos específica. Cada base de datos tiene su propio proveedor, pero todos funcionan igual.

Base de datos	Proveedor	Clases
SQL Server	`Microsoft.Data.SqlClient`	`SqlConnection, SqlCommand`
MySQL	`MySql.Data`	`MySqlConnection, MySqlCommand`
PostgreSQL	`Npgsql`	`NpgsqlConnection, NpgsqlCommand`

Se instala con NuGet:

```
dotnet add package Microsoft.Data.SqlClient
```

3. Connection

Es el objeto que abre el "camino" entre el backend y la base de datos. Necesita una **cadena de conexión**, que dice dónde está el servidor, cuál es la base de datos y cómo entrar.

```
string cadena = "Server=localhost;Database=Escuela;Trusted_Connection=True;";

using SqlConnection conexion = new SqlConnection(cadena);
conexion.Open();
```

Explicación: `Open()` abre la conexión. El `using` la cierra automáticamente al terminar, aunque haya un error. Esto es importante porque las conexiones abiertas gastan recursos.

4. Command

Es el objeto que lleva la instrucción SQL a la base de datos. Usa la conexión para mandarla.

```
SqlCommand comando = new SqlCommand("SELECT * FROM Alumnos", conexion);
```

Formas de ejecutarlo:

- `ExecuteReader()`: para consultas que regresan varios registros (SELECT).
- `ExecuteNonQuery()`: para INSERT, UPDATE y DELETE. Regresa cuántas filas cambiaron.
- `ExecuteScalar()`: cuando solo se necesita un valor, por ejemplo SELECT COUNT(*).

```
int total = (int)new SqlCommand("SELECT COUNT(*) FROM Alumnos", conexion).ExecuteScalar();
```

5. DataReader

Lee los resultados de una consulta **fila por fila, hacia adelante**, como leer una lista de arriba hacia abajo sin poder regresar. Es muy rápido, pero la conexión debe estar abierta mientras se lee.

```
using SqlDataReader lector = comando.ExecuteReader();

while (lector.Read())
{
    int id = lector.GetInt32(0);
    string nombre = lector.GetString(1);
    Console.WriteLine($"{id} - {nombre}");
}
```

Explicación: `Read()` avanza a la siguiente fila y regresa false cuando ya no hay más. Con `GetInt32` y `GetString` se obtiene el valor de cada columna.

6. DataAdapter y DataSet

- **DataSet:** es como una copia de la base de datos en la memoria. Puede tener varias tablas (DataTable).
- **DataAdapter:** es el "puente" que llena el DataSet con los datos de la base y también puede regresar los cambios.

Se usan en el modo **desconectado**: el adaptador abre la conexión, copia los datos y la cierra solo.

```
SqlDataAdapter adaptador = new SqlDataAdapter("SELECT * FROM Alumnos", conexion);
DataSet datos = new DataSet();
adaptador.Fill(datos, "Alumnos");

foreach (DataRow fila in datos.Tables["Alumnos"].Rows)
{
    Console.WriteLine(fila["Nombre"]);
}
```

Explicación: `Fill()` ejecuta la consulta y guarda los resultados en el DataSet. Después se pueden recorrer las filas sin necesidad de la conexión.

Diferencia: el DataReader es más rápido y gasta menos memoria. El DataSet permite trabajar con los datos sin estar conectado.

7. Parámetros SQL

Sirven para mandar valores a una consulta **de forma segura**. En lugar de pegar el texto del usuario dentro del SQL, se usa un "espacio" (`@nombre`) y el valor se manda aparte.

Esto evita la **inyección SQL**, que pasa cuando alguien escribe código SQL en un campo para robar o borrar datos.

Mal (inseguro):

```
string sql = "SELECT * FROM Alumnos WHERE Nombre = '" + nombre + "'";
```

Bien (con parámetros):

```
SqlCommand comando = new SqlCommand(
    "INSERT INTO Alumnos (Nombre, Edad) VALUES (@nombre, @edad)", conexion);

comando.Parameters.AddWithValue("@nombre", "Ana");
comando.Parameters.AddWithValue("@edad", 20);

int filas = comando.ExecuteNonQuery();
```

Explicación: `@nombre` y `@edad` son los espacios para los valores. Con `Parameters.AddWithValue` se les da su valor, y ADO.NET se encarga de que se traten como datos y no como código.

8. Excepciones

Son los errores que pueden pasar al trabajar con la base de datos. Por ejemplo, que el servidor esté apagado, que la contraseña sea incorrecta, que la tabla no exista o que se repita una llave primaria. Se manejan con **try / catch** para que la aplicación no se cierre y se muestre un mensaje claro.

```
try
{
    using SqlConnection conexion = new SqlConnection(cadena);
    conexion.Open();
    // ejecutar comandos
}
catch (SqlException ex)
{
    Console.WriteLine("Error de base de datos: " + ex.Message);
}
catch (Exception ex)
{
    Console.WriteLine("Error general: " + ex.Message);
}
finally
{
    Console.WriteLine("Fin del proceso");
}
```

- **try:** el código que puede fallar.
- **catch (SqlException):** atrapa errores propios de SQL Server.
- **catch (Exception):** atrapa cualquier otro error.
- **finally:** se ejecuta siempre, falle o no.

Ejemplo completo del flujo

```
try
{
    using SqlConnection conexion = new SqlConnection(cadena);     // 1. Conexión
    conexion.Open();

    SqlCommand comando = new SqlCommand(                          // 2. Comando
        "SELECT Id, Nombre FROM Alumnos WHERE Edad > @edad", conexion);
    comando.Parameters.AddWithValue("@edad", 18);                 // 3. Parámetro

    using SqlDataReader lector = comando.ExecuteReader();         // 4. Ejecutar
    while (lector.Read())                                         // 5. Leer
    {
        Console.WriteLine(lector["Nombre"]);
    }
}                                                                 // 6. Se cierra sola
catch (SqlException ex)                                           // 7. Error
{
    Console.WriteLine(ex.Message);
}
```